Minimum Spanning Tree with Neighborhoods

Heuristic Approach

Arthur Péraud

Supervised by Prof. Fatiha Bendali and Dr. Jean Mailfert

01/12/2025

Agenda

Introduction and Motivation

Theoretical Foundations

Heuristic Algorithm

Experimental Results

Future Work

Kruskal's Algorithm

Overview

Finds minimum spanning tree by greedily selecting edges in order of increasing weight, skipping edges that would create cycles.

Steps:

1. Sort all edges by weight
2. Initialize each vertex as its own component
3. For each edge in sorted order: add if no cycle
4. Stop when $n - 1$ edges are selected

Complexity: $O(|E| \log |E|)$
Why: Used repeatedly in our heuristic

Motivation: Why MSTN?

Classical MST Problem:

- Vertices at *fixed* known locations
- Connect with minimum total edge length
- Solved efficiently in polynomial time

Real-World Challenge:

- Terminal positions *uncertain* or *flexible*
- Geometric tolerances, spatial constraints
- Each terminal occupies a *region*

Applications:

1. Network design
2. Facility location
3. Telecom networks
4. Supply chain
5. Logistics

Classical MST vs MSTN

Aspect	MST	MSTN
Positions	Fixed	Neighborhoods
Complexity	Polynomial	NP-hard
Approach	Greedy	Heuristic/MINLP

Key Insight: Place representatives strategically to reduce tree length

Problem Definition

Given:

- Graph $G = (V, E)$ with n vertices
- Each vertex has convex neighborhood N_v

Minimize:

$$\min_{y,T} \sum_{(i,j)\in T} \|y_i - y_j\|$$

where $y_v \in N_v$ and T is spanning tree

Key Property: MSTN $\leq$ MST

Fundamental Observation:
Let c_v = center of N_v

$$\text{MSTN}^* \leq \text{MST}(\{c_1, \ldots, c_n\})$$

Why: Centers are feasible solution $\Rightarrow$ optimizer can do at least as well

Practical Impact:

- Large neighborhoods $\Rightarrow$ representatives move closer
- Achievable reduction: **30–40%**

Biconvex Structure

Key Observation: MSTN is **biconvex**

Scenario	**Property**
Fix tree T	Convex problem in y
Fix positions Y	Easy: classical MST
Both together	Non-convex

Solution: Alternate optimization

1. Compute MST on current positions
2. Update positions (convex subproblem)
3. Repeat until convergence

Heuristic Algorithm Overview

Main Idea: Alternate between two steps

Initialize at centers

$\Downarrow$

Compute MST (Kruskal)

$\Downarrow$

Move toward tree neighbors

$\Downarrow$

Converged?

YES $\rightarrow$ Return

NO $\rightarrow$ Repeat

Key: Move representatives closer to tree neighbors, stay in neighborhoods

Algorithm Details

Initialization:

- Set $y_i^0 =$ center of N_i
- Compute MST, record baseline cost C_{orig}

Iteration k:

1. Compute MST T^k on current positions
2. For each vertex i: find neighbors in T^k
3. Compute barycenter: $b_i^k = \frac{1}{|N_i|} \sum_{j \sim i} y_j^k$
4. Project onto neighborhood: $y_i^{k+1} = \text{Proj}_{N_i}(b_i^k)$
5. Check convergence

Projection onto Square Neighborhood

Setup: Vertex i has axis-aligned square neighborhood N_i with:

- Center: $C_i = (c_x, c_y)$
- Side length: s_i
- Bounds: $x \in [x_{\min}, x_{\max}]$, $y \in [y_{\min}, y_{\max}]$ where:

$$x_{\min} = c_x - \frac{s_i}{2}, \quad x_{\max} = c_x + \frac{s_i}{2}, \quad y_{\min} = c_y - \frac{s_i}{2}, \quad y_{\max} = c_y + \frac{s_i}{2}$$

For barycenter $B^{(k)} = (b_x, b_y)$, we solve independently for each coordinate:

$$\min_{p_x \in [x_{\min}, x_{\max}]} (b_x - p_x)^2, \qquad \min_{p_y \in [y_{\min}, y_{\max}]} (b_y - p_y)^2$$

Projection onto Square Neighborhood

$$\text{Proj}_{N_i}(b_x, b_y) = (\min(\max(b_x, x_{\min}), x_{\max}), \min(\max(b_y, y_{\min}), y_{\max}))$$

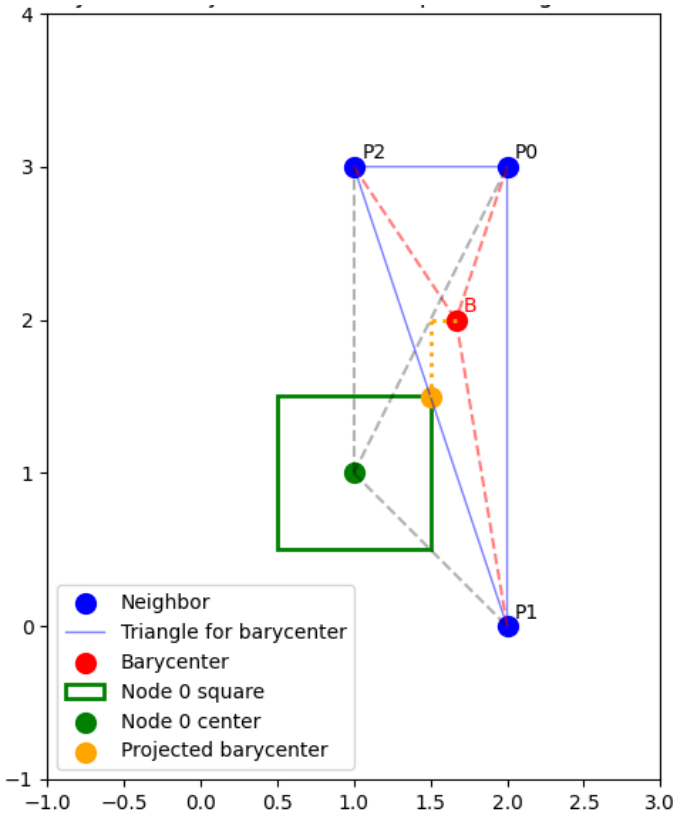

Fig 0: Projection of B onto N_i

Single Instance Example (35 neighborhoods)

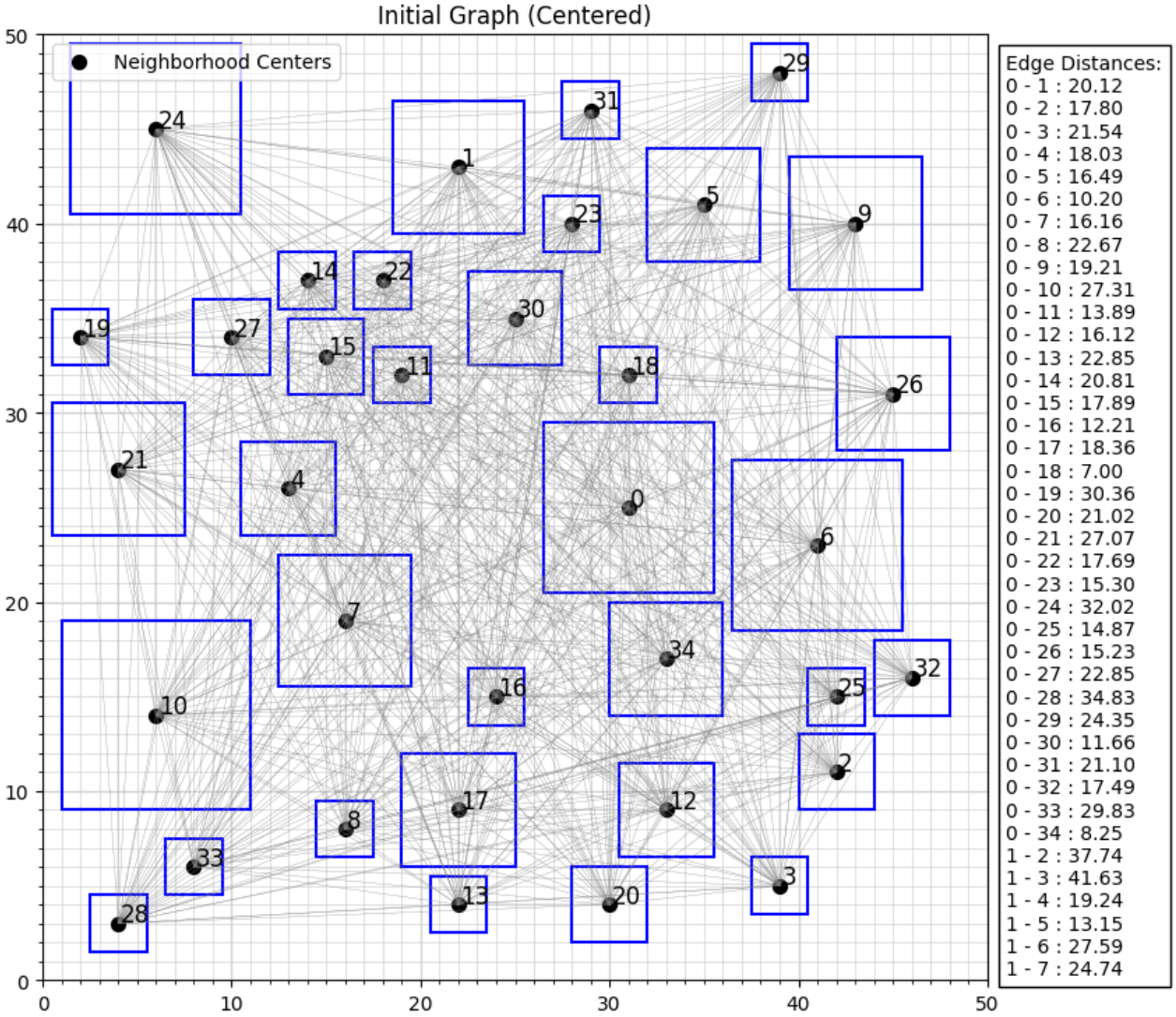

Fig 1

Single Instance Example (35 neighborhoods)

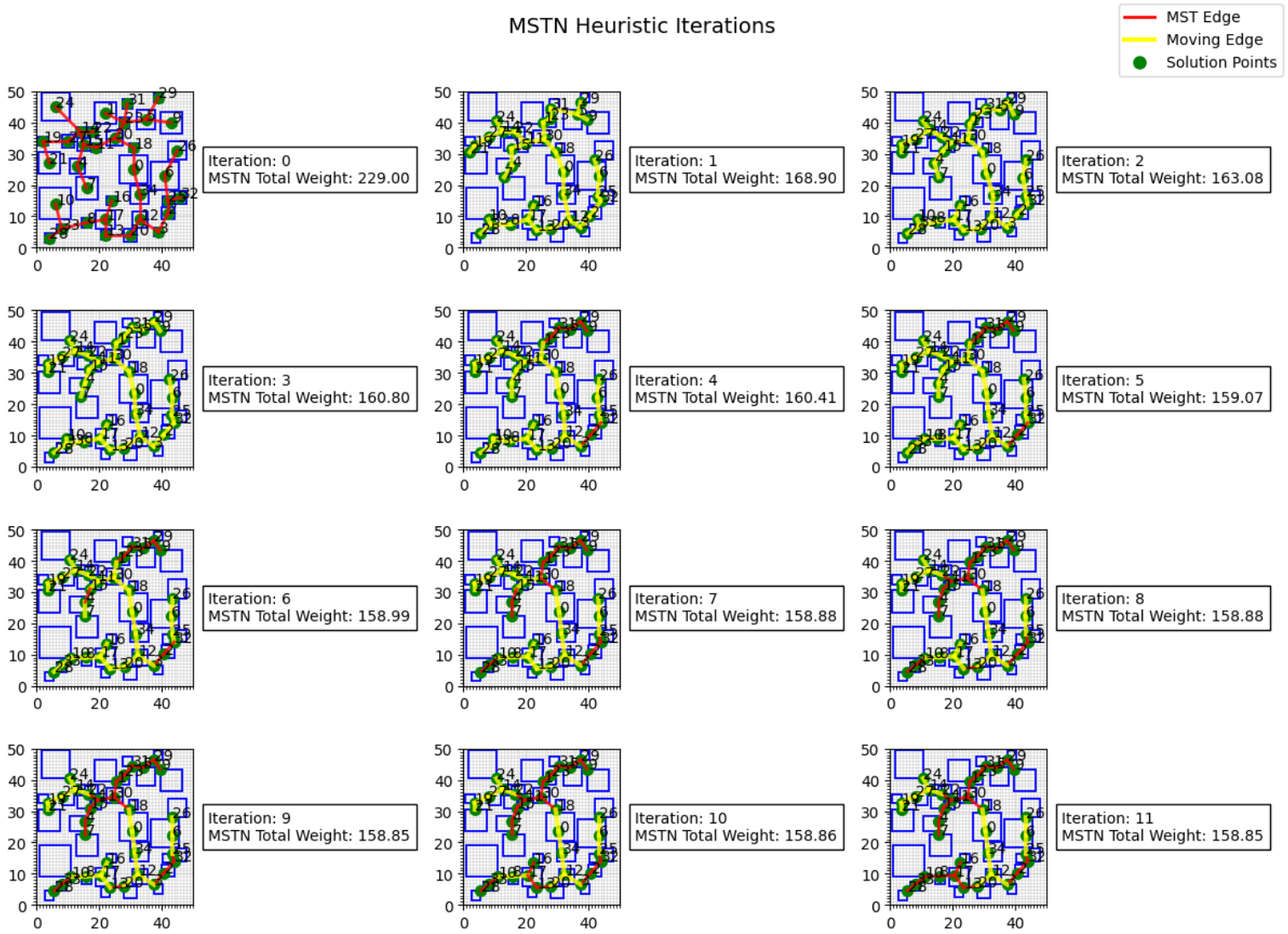

Fig 2

Single Instance Example (35 neighborhoods)

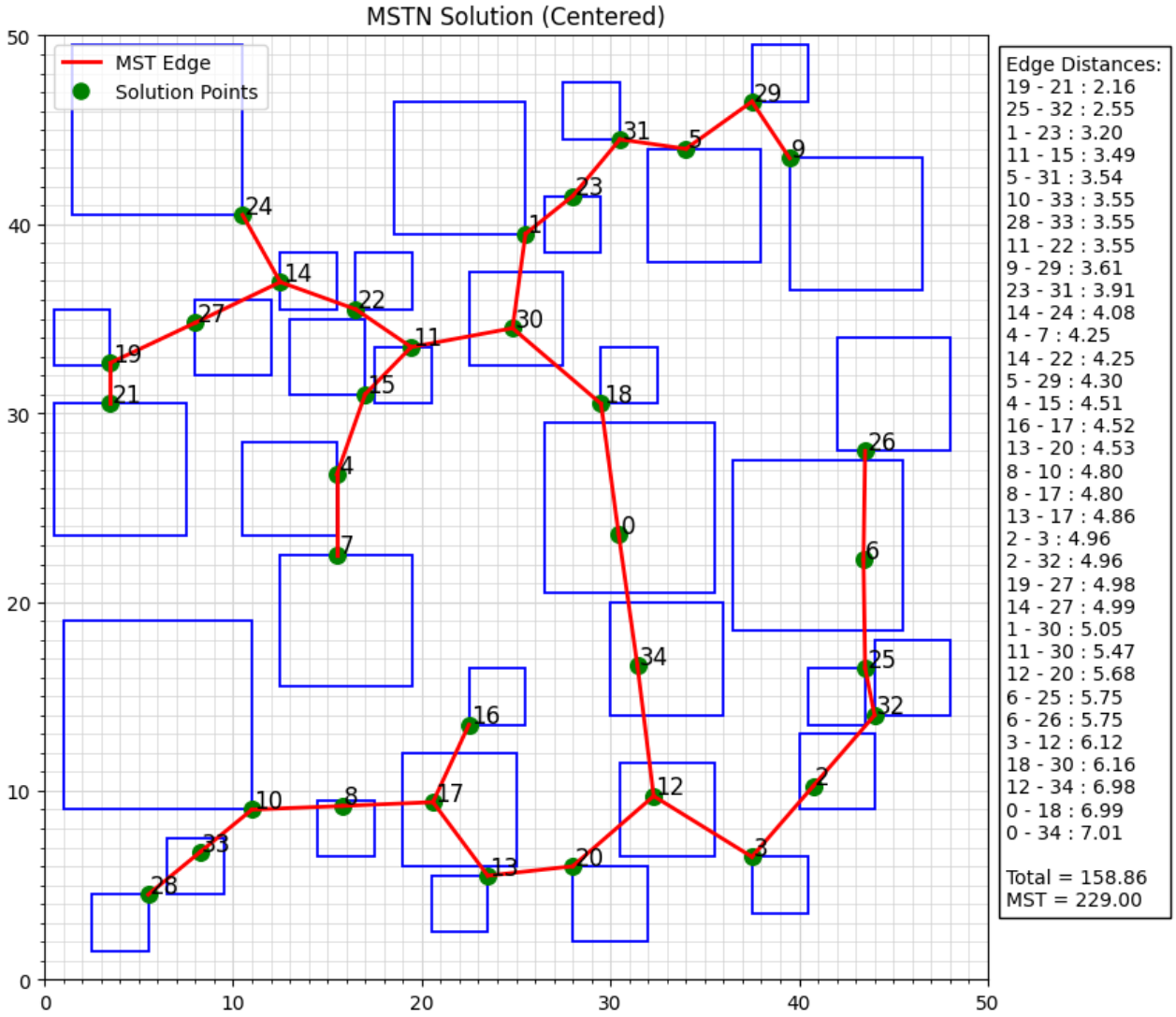

Fig 3

Single Instance Example (35 neighborhoods)

Setup:

- 35 square neighborhoods, complete graph
- Integer, non-overlapping

Initial cost	229.00
After 1 iter.	168.90
Final cost	158.86
Reduction	**30.6%**

Fast convergence, most improvement in first iterations

Cost Reduction Across Many Instances

Experimental Setup:

- Integer, non-overlapping
- Grid sizes: 1 to 865
- Number of neighborhoods: $n = 0.8 \times$ grid size
- Neighborhood sizes: random, in $[0.01, 0.03] \times$ grid size

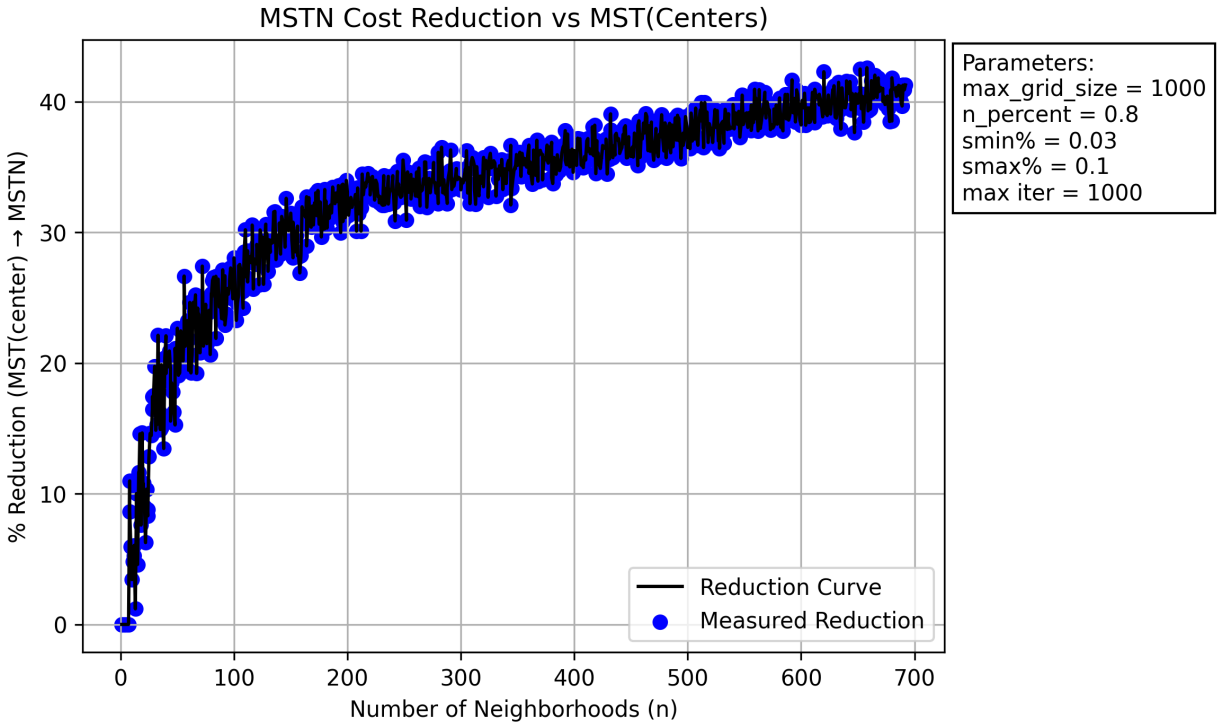

Fig 4

Scalability Results

Performance as n grows:

Iterations:

- Moderate growth with n
- Typically 5–15 iterations
- Efficient updates

Time Complexity:

- Per iteration: $O(|E| \log |E|)$
- Total: $O(K \cdot |E| \log |E)|$
- Practical for 100s of vertices

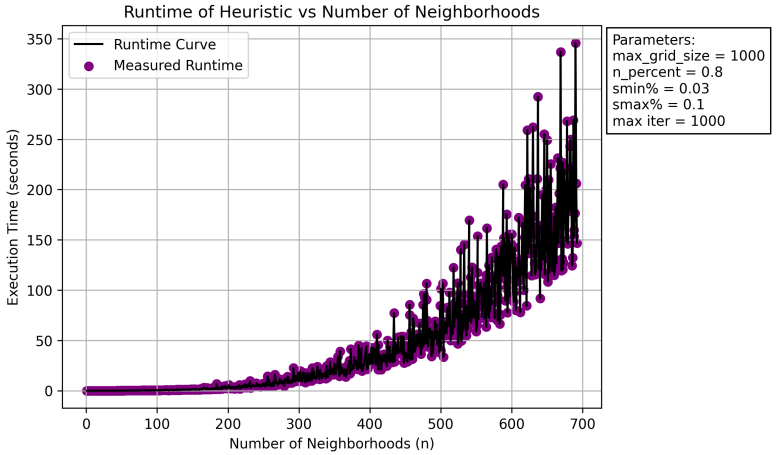

Fig 5

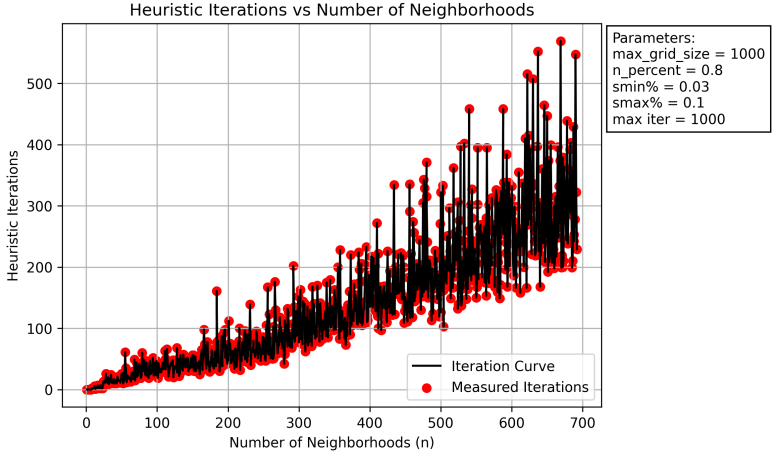

Fig 6

Why Heuristic? Heuristic vs. Exact Methods

Exact (MINLP):

- Optimal solutions
- Only $n \lesssim 50$
- Expensive

Our Heuristic:

- Scales to 100s
- Very fast
- 30–40% improvement

Trade-off: Speed + Scalability vs. Optimality Guarantee

Future Work Roadmap

Three Research Directions:

1. **MINLP Formulations**
   - Exact methods for validation
   - SEC vs. MTZ approaches
   - Benders decomposition
2. **Advanced Heuristics**
   - Multi-start strategies
   - Local search in tree space
   - Adaptive step sizes
3. **Machine Learning**
   - Learn update rules
   - Neural tree construction
   - Meta-learning

MINLP Formulations

Why Exact Methods?

Provide optimality guarantees for small instances and validate heuristic quality

Two Main Approaches:

- **SEC (Subtour Elimination):** Stronger bounds, better performance
- **MTZ (Miller-Tucker-Zemlin):** Compact, weaker bounds

Limitation: Both face scalability limits for $n > 50$

RL–MSTN: MINLP Formulation (1/2)

Decision variables:

- $y_v \in N_v \subset \mathbb{R}^d$ for each $v \in V$ (representative in its neighborhood)
- $x_e \in \{0, 1\}$ for each $e \in E$ (1 if edge e is in the tree)
- $u_e \geq 0$ for each $e = \{i, j\} \in E$ (edge length)

Nonlinear link:

$$u_{\{i,j\}} \;\geq\; \|y_i - y_j\| \quad \forall e = \{i, j\} \in E$$

To **linearize the objective**, introduce:

- $\theta_e = u_e x_e$,
- an upper bound $\bar{U}_e$ on u_e,

so the objective becomes

$$\min \sum_{e \in E} \theta_e.$$

The product $\theta_e = u_e x_e$ is enforced with McCormick inequalities:

$$\theta_e \leq \bar{U}_e x_e \tag{U1}$$

RL–MSTN: MINLP Formulation (2/2)

Let $ST(G)$ be the spanning tree polytope of G.
RL–MSTN model:

$$\min \sum_{e \in E} \theta_e$$

subject to

$$\begin{aligned}
& x \in ST(G), && \\
& y_v \in N_v, && \forall v \in V, \\
& u_{\{i,j\}} \geq \|y_i - y_j\|, && \forall \{i,j\} \in E, \\
& \theta_e \leq \bar{U}_e x_e, && \forall e \in E, \\
& \theta_e \geq u_e - \bar{U}_e(1 - x_e), && \forall e \in E, \\
& \theta_e \leq u_e,\ \theta_e \geq 0,\ u_e \geq 0, && \forall e \in E, \\
& x_e \in \{0,1\}, && \forall e \in E.
\end{aligned}$$

Advanced Heuristics: Multi-Start & Local Search

Multi-Start Strategies:

- Initialize from multiple different starting trees
- Run heuristic from each start
- Keep best solution found
- Potential improvement

Local Search in Tree Space:

- After convergence, modify tree structure
- Try edge swaps (2-opt moves)
- Reoptimize positions for new tree
- Escape poor local minima

Advanced Heuristics: Adaptive Methods

Adaptive Step Sizes:

- Partial steps instead of full moves
- $y_i^{k+1} = y_i^k + \alpha_k(\text{Proj}(b_i^k) - y_i^k)$
- Improves stability and convergence ?

Variable Neighborhood Descent:

- Combine alternation with tree modifications
- Broader solution space exploration
- Higher cost but better solutions ?

Machine Learning for Heuristic Design

Three ML Approaches:

1. **Learn Update Rules**
   - Supervised: learn optimal movement from data
   - Reinforcement: optimize policy via trial-and-error
2. **Learn Tree Construction**
   - Graph neural networks (GNNs)
   - Build good initial trees
3. **Meta-Learning**
   - Predict heuristic performance on new instances
   - Guide algorithm selection

Conclusion

Summary:

- MSTN: challenging geometric network design problem
- Biconvex structure enables alternating heuristics
- Simple, fast heuristic: 30–40% cost reduction
- Scales to hundreds of neighborhoods

Thank You!